

How will the job be done technically and managerially? Once product scope is established, a management and technical strategy for the project must be defined.

How much of each resource is needed? The answer to this question is derived by developing estimates (Chapter 26) based on answers to earlier questions.

Boehm's W⁵HH Principle is applicable regardless of the size or complexity of a software project. The questions noted provide you and your team with an excellent planning outline.

24.7 CRITICAL PRACTICES

The Airlie Council⁸ has developed a list of "critical software practices for performance-based management." These practices are "consistently used by, and considered critical by, highly successful software projects and organizations whose 'bottom line' performance is consistently much better than industry averages" [Air99].

Critical practices⁹ include: metric-based project management (Chapter 25), empirical cost and schedule estimation (Chapters 26 and 27), earned value tracking (Chapter 27), defect tracking against quality targets (Chapters 14 through 16), and people aware management (Section 24.2). Each of these critical practices is addressed throughout Parts 3 and 4 of this book.

SOFTWARE TOOLS



Software Tools for Project Managers

The "tools" listed here are generic and apply to a broad range of activities performed by project managers. Specific project management tools (e.g., scheduling tools, estimating tools, risk analysis tools) are considered in later chapters.

Representative Tools:¹⁰

The Software Program Manager's Network (www.spmn.com) has developed a simple tool called *Project Control Panel*, which provides project managers with an direct indication of project status.

The tool has "gauges" much like a dashboard and is implemented with Microsoft Excel. It is available for download at www.spmn.com/products_software.html.

Ganthead.com (www.ganthead.com/) has developed a set of useful *checklists for project managers*.

Ittoolkit.com (www.ittoolkit.com) provides "a collection of planning guides, process templates and smart worksheets" available on CD-ROM.

⁸ The Airlie Council was comprised of a team of software engineering experts chartered by the U.S. Department of Defense to help develop guidelines for best practices in software project management and software engineering. For more on best practices, see www.swqual.com/newsletter/vol1/no3/vol1no3.html.

⁹ Only those critical practices associated with "project integrity" are noted here.

¹⁰ Tools noted here do not represent an endorsement, but rather a sampling of tools in this category. In most cases, tool names are trademarked by their respective developers.

24.8 SUMMARY

Software project management is an umbrella activity within software engineering. It begins before any technical activity is initiated and continues throughout the modeling, construction, and deployment of computer software.

Four P's have a substantial influence on software project management—people, product, process, and project. People must be organized into effective teams, motivated to do high-quality software work, and coordinated to achieve effective communication. Product requirements must be communicated from customer to developer, partitioned (decomposed) into their constituent parts, and positioned for work by the software team. The process must be adapted to the people and the product. A common process framework is selected, an appropriate software engineering paradigm is applied, and a set of work tasks is chosen to get the job done. Finally, the project must be organized in a manner that enables the software team to succeed.

The pivotal element in all software projects is people. Software engineers can be organized in a number of different team structures that range from traditional control hierarchies to “open paradigm” teams. A variety of coordination and communication techniques can be applied to support the work of the team. In general, technical reviews and informal person-to-person communication have the most value for practitioners.

The project management activity encompasses measurement and metrics, estimation and scheduling, risk analysis, tracking, and control. Each of these topics is considered in the chapters that follow.

PROBLEMS AND POINTS TO PONDER

24.1. Based on information contained in this chapter and your own experience, develop “ten commandments” for empowering software engineers. That is, make a list of 10 guidelines that will lead to software people who work to their full potential.

24.2. The Software Engineering Institute's People Capability Maturity Model (People-CMM) takes an organized look at “key practice areas” that cultivate good software people. Your instructor will assign you one KPA for analysis and summary.

24.3. Describe three real-life situations in which the customer and the end user are the same. Describe three situations in which they are different.

24.4. The decisions made by senior management can have a significant impact on the effectiveness of a software engineering team. Provide five examples to illustrate that this is true.

24.5. Review a copy of Weinberg's book [Wei86], and write a two- or three-page summary of the issues that should be considered in applying the MOI model.

24.6. You have been appointed a project manager within an information systems organization. Your job is to build an application that is quite similar to others your team has built, although this one is larger and more complex. Requirements have been thoroughly documented by the customer. What team structure would you choose and why? What software process model(s) would you choose and why?

24.7. You have been appointed a project manager for a small software products company. Your job is to build a breakthrough product that combines virtual reality hardware with state-of-the-art software. Because competition for the home entertainment market is intense, there is significant pressure to get the job done. What team structure would you choose and why? What software process model(s) would you choose and why?

24.8. You have been appointed a project manager for a major software products company. Your job is to manage the development of the next-generation version of its widely used word-processing software. Because competition is intense, tight deadlines have been established and announced. What team structure would you choose and why? What software process model(s) would you choose and why?

24.9. You have been appointed a software project manager for a company that services the genetic engineering world. Your job is to manage the development of a new software product that will accelerate the pace of gene typing. The work is R&D oriented, but the goal is to produce a product within the next year. What team structure would you choose and why? What software process model(s) would you choose and why?

24.10. You have been asked to develop a small application that analyzes each course offered by a university and reports the average grade obtained in the course (for a given term). Write a statement of scope that bounds this problem.

24.11. Do a first-level functional decomposition of the page layout function discussed briefly in Section 24.3.2.

FURTHER READINGS AND INFORMATION SOURCES

The Project Management Institute (*Guide to the Project Management Body of Knowledge*, PMI, 2001) covers all important aspects of project management. Bechtold (*Essentials of Software Project Management*, 2d ed., Management Concepts, 2007), Wysocki (*Effective Software Project Management*, Wiley, 2006), Stelman and Greene (*Applied Software Project Management*, O'Reilly, 2005), and Berkun (*The Art of Project Management*, O'Reilly, 2005) teach basic skills and provide detailed guidance for all software project management tasks. McConnell (*Professional Software Development*, Addison-Wesley, 2004) offers pragmatic advice for achieving "shorter schedules, higher quality products, and more successful projects." Henry (*Software Project Management*, Addison-Wesley, 2003) offers real-world advice that is useful for all project managers.

Tom DeMarco and his colleagues (*Adrenaline Junkies and Template Zombies*, Dorset House, 2008) have written an insightful treatment of the human patterns that are encountered in every software project. An excellent four-volume series written by Weinberg (*Quality Software Management*, Dorset House, 1992, 1993, 1994, 1996) introduces basic systems thinking and management concepts, explains how to use measurements effectively, and addresses "congruent action," the ability to establish "fit" between the manager's needs, the needs of technical staff, and the needs of the business. It will provide both new and experienced managers with useful information. Futrell and his colleagues (*Quality Software Project Management*, Prentice-Hall, 2002) present a voluminous treatment of project management. Brown and his colleagues (*Antipatterns in Project Management*, Wiley, 2000) discuss what not to do during the management of a software project.

Brooks (*The Mythical Man-Month*, Anniversary Edition, Addison-Wesley, 1995) has updated his classic book to provide new insight into software project and management issues. McConnell (*Software Project Survival Guide*, Microsoft Press, 1997) presents excellent pragmatic guidance for those who must manage software projects. Purba and Shah (*How to Manage a Successful Software Project*, 2d ed., Wiley, 2000) present a number of case studies that indicate why some projects succeed and others fail. Bennatan (*On Time Within Budget*, 3d ed., Wiley, 2000) presents useful tips and guidelines for software project managers. Weigers (*Practical Project Initiation*, Microsoft Press, 2007) provides practical guidelines for getting a software project off the ground successfully.

It can be argued that the most important aspect of software project management is people management. Cockburn (*Agile Software Development*, Addison-Wesley, 2002) presents one of

the best discussions of software people written to date. DeMarco and Lister [DeM98] have written the definitive book on software people and software projects. In addition, the following books on this subject have been published in recent years and are worth examining:

- Cantor, M., *Software Leadership: A Guide to Successful Software Development*, Addison-Wesley, 2001.
- Carmel, E., *Global Software Teams: Collaborating Across Borders and Time Zones*, Prentice Hall, 1999.
- Constantine, L., *Peopleware Papers: Notes on the Human Side of Software*, Prentice Hall, 2001.
- Garton, C., and K. Wegryn, *Managing Without Walls*, McPress, 2006.
- Humphrey, W. S., *Managing Technical People: Innovation, Teamwork, and the Software Process*, Addison-Wesley, 1997.
- Humphrey, W. S., *TSP-Coaching Development Teams*, Addison-Wesley, 2006.
- Jones, P. H., *Handbook of Team Design: A Practitioner's Guide to Team Systems Development*, McGraw-Hill, 1997.
- Karolak, D. S., *Global Software Development: Managing Virtual Teams and Environments*, IEEE Computer Society, 1998.
- Peters, L., *Getting Results from Software Development Teams*, Microsoft Press, 2008.
- Whitehead, R., *Leading a Software Development Team*, Addison-Wesley, 2001.

Even though they do not relate specifically to the software world and sometimes suffer from oversimplification and broad generalization, best-selling “management” books by Kanter (*Confidence*, Three Rivers Press, 2006), Covey (*The 8th Habit*, Free Press, 2004), Bossidy (*Execution: The Discipline of Getting Things Done*, Crown Publishing, 2002), Drucker (*Management Challenges for the 21st Century*, Harper Business, 1999), Buckingham and Coffman (*First, Break All the Rules: What the World's Greatest Managers Do Differently*, Simon and Schuster, 1999), and Christensen (*The Innovator's Dilemma*, Harvard Business School Press, 1997) emphasize “new rules” defined by a rapidly changing economy. Older titles such as *Who Moved My Cheese?*, *The One-Minute Manager*, and *In Search of Excellence* continue to provide valuable insights that can help you to manage people and projects more effectively.

A wide variety of information sources on the software project management are available on the Internet. An up-to-date list of World Wide Web references relevant to software project management can be found at the SEPA website: **www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm**.

**KEY
CONCEPTS**

defect removal
efficiency
(DRE)681

function point . .673

measurement . .671

metrics,667

arguments
for683

baseline683

establishing
a program ...686

function-
oriented673

LOC-based ...675

object-
oriented675

Measurement enables us to gain insight into the process and the project by providing a mechanism for objective evaluation. Lord Kelvin once said:

When you can measure what you are speaking about and express it in numbers, you know something about it; but when you cannot measure, when you cannot express it in numbers, your knowledge is of a meager and unsatisfactory kind: it may be the beginning of knowledge, but you have scarcely, in your thoughts, advanced to the stage of a science.

The software engineering community has taken Lord Kelvin's words to heart. But not without frustration and more than a little controversy!

Measurement can be applied to the software process with the intent of improving it on a continuous basis. Measurement can be used throughout a software project to assist in estimation, quality control, productivity assessment, and project control. Finally, measurement can be used by software engineers to help assess the quality of work products and to assist in tactical decision making as a project proceeds (Chapter 23).

**QUICK
LOOK**

What is it? Software process and project metrics are quantitative measures that enable you to gain insight into the efficacy of the software process and the projects that are conducted using the process as a framework. Basic quality and productivity data are collected. These data are then analyzed, compared against past averages, and assessed to determine whether quality and productivity improvements have occurred. Metrics are also used to pinpoint problem areas so that remedies can be developed and the software process can be improved.

Who does it? Software metrics are analyzed and assessed by software managers. Measures are often collected by software engineers.

Why is it important? If you don't measure, judgment can be based only on subjective evaluation.

With measurement, trends (either good or bad) can be spotted, better estimates can be made, and true improvement can be accomplished over time.

What are the steps? Begin by defining a limited set of process, project, and product measures that are easy to collect. These measures are often normalized using either size- or function-oriented metrics. The result is analyzed and compared to past averages for similar projects performed within the organization. Trends are assessed and conclusions are generated.

What is the work product? A set of software metrics that provide insight into the process and understanding of the project.

How do I ensure that I've done it right? By applying a consistent, yet simple measurement scheme that is never to be used to assess, reward, or punish individual performance.

process	667
productivity ..	675
project	670
public and private	668
size-oriented ..	672
software quality	679
use-case- oriented	676
WebApp	677

Within the context of the software process and the projects that are conducted using the process, a software team is concerned primarily with productivity and quality metrics—measures of software development “output” as a function of effort and time applied and measures of the “fitness for use” of the work products that are produced. For planning and estimating purposes, your interest is historical. What was software development productivity on past projects? What was the quality of the software that was produced? How can past productivity and quality data be extrapolated to the present? How can it help you plan and estimate more accurately?

In their guidebook on software measurement, Park, Goethert, and Florac [Par96b] note the reasons that we measure: (1) to *characterize* in an effort to gain an understanding “of processes, products, resources, and environments, and to establish baselines for comparisons with future assessments”; (2) to *evaluate* “to determine status with respect to plans”; (3) to *predict* by “gaining understandings of relationships among processes and products and building models of these relationships”; and (4) to *improve* by “identify[ing] roadblocks, root causes, inefficiencies, and other opportunities for improving product quality and process performance.”

Measurement is a management tool. If conducted properly, it provides a project manager with insight. And as a result, it assists the project manager and the software team in making decisions that will lead to a successful project.

25.1 METRICS IN THE PROCESS AND PROJECT DOMAINS

KEY POINT

Process metrics have long-term impact. Their intent is to improve the process itself. Project metrics often contribute to the development of process metrics.

Process metrics are collected across all projects and over long periods of time. Their intent is to provide a set of process indicators that lead to long-term software process improvement. *Project metrics* enable a software project manager to (1) assess the status of an ongoing project, (2) track potential risks, (3) uncover problem areas before they go “critical,” (4) adjust work flow or tasks, and (5) evaluate the project team’s ability to control quality of software work products.

Measures that are collected by a project team and converted into metrics for use during a project can also be transmitted to those with responsibility for software process improvement (Chapter 30). For this reason, many of the same metrics are used in both the process and project domains.

25.1.1 Process Metrics and Software Process Improvement

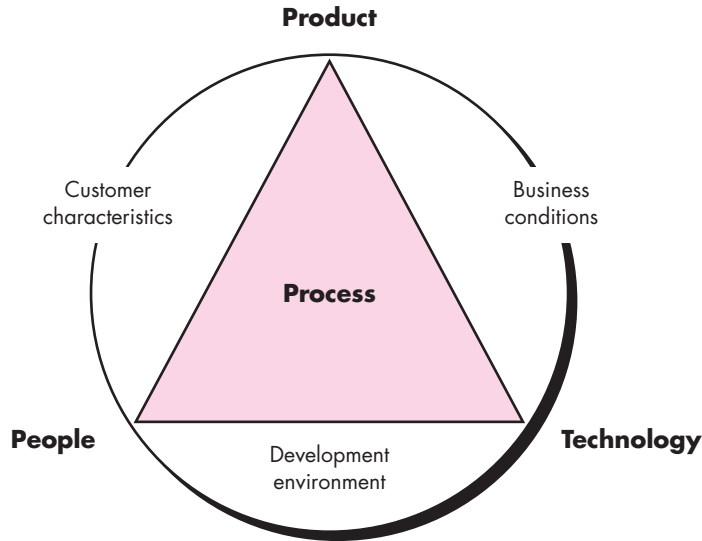
The only rational way to improve any process is to measure specific attributes of the process, develop a set of meaningful metrics based on these attributes, and then use the metrics to provide indicators that will lead to a strategy for improvement (Chapter 30). But before I discuss software metrics and their impact on software process improvement, it is important to note that process is only one of a number of “controllable factors in improving software quality and organizational performance” [Pau94].

Referring to Figure 25.1, process sits at the center of a triangle connecting three factors that have a profound influence on software quality and organizational

FIGURE 25.1

Determinants for software quality and organizational effectiveness.

Source: Adapted from [Pau94].



KEY POINT

The skill and motivation of the software people doing the work are the most important factors that influence software quality.

performance. The skill and motivation of people has been shown [Boe81] to be the single most influential factor in quality and performance. The complexity of the product can have a substantial impact on quality and team performance. The technology (i.e., the software engineering methods and tools) that populates the process also has an impact.

In addition, the process triangle exists within a circle of environmental conditions that include the development environment (e.g., integrated software tools), business conditions (e.g., deadlines, business rules), and customer characteristics (e.g., ease of communication and collaboration).

You can only measure the efficacy of a software process indirectly. That is, you derive a set of metrics based on the outcomes that can be derived from the process. Outcomes include measures of errors uncovered before release of the software, defects delivered to and reported by end users, work products delivered (productivity), human effort expended, calendar time expended, schedule conformance, and other measures. You can also derive process metrics by measuring the characteristics of specific software engineering tasks. For example, you might measure the effort and time spent performing the umbrella activities and the generic software engineering activities described in Chapter 2.

Grady [Gra92] argues that there are “private and public” uses for different types of process data. Because it is natural that individual software engineers might be sensitive to the use of metrics collected on an individual basis, these data should be private to the individual and serve as an indicator for the individual only. Examples of *private metrics* include defect rates (by individual), defect rates (by component), and errors found during development.

note:

“Software metrics let you know when to laugh and when to cry.”

Tom Gilb

? What is the difference between private and public uses for software metrics?

The “private process data” philosophy conforms well with the Personal Software Process approach (Chapter 2) proposed by Humphrey [Hum97]. Humphrey recognizes that software process improvement can and should begin at the individual level. Private process data can serve as an important driver as you work to improve your software engineering approach.

Some process metrics are private to the software project team but public to all team members. Examples include defects reported for major software functions (that have been developed by a number of practitioners), errors found during technical reviews, and lines of code or function points per component or function.¹ The team reviews these data to uncover indicators that can improve team performance.

Public metrics generally assimilate information that originally was private to individuals and teams. Project-level defect rates (absolutely not attributed to an individual), effort, calendar times, and related data are collected and evaluated in an attempt to uncover indicators that can improve organizational process performance.

Software process metrics can provide significant benefit as an organization works to improve its overall level of process maturity. However, like all metrics, these can be misused, creating more problems than they solve. Grady [Gra92] suggests a “software metrics etiquette” that is appropriate for both managers and practitioners as they institute a process metrics program:

- Use common sense and organizational sensitivity when interpreting metrics data.
- Provide regular feedback to the individuals and teams who collect measures and metrics.
- Don’t use metrics to appraise individuals.
- Work with practitioners and teams to set clear goals and metrics that will be used to achieve them.
- Never use metrics to threaten individuals or teams.
- Metrics data that indicate a problem area should not be considered “negative.” These data are merely an indicator for process improvement.
- Don’t obsess on a single metric to the exclusion of other important metrics.

As an organization becomes more comfortable with the collection and use of process metrics, the derivation of simple indicators gives way to a more rigorous approach called *statistical software process improvement* (SSPI). In essence, SSPI uses software failure analysis to collect information about all errors and defects² encountered as an application, system, or product is developed and used.

? What guidelines should be applied when we collect software metrics?

¹ Lines of code and function point metrics are discussed in Sections 25.2.1 and 25.2.2.

² In this book, an *error* is defined as some flaw in a software engineering work product that is uncovered *before* the software is delivered to the end user. A *defect* is a flaw that is uncovered *after* delivery to the end user. It should be noted that others do not make this distinction.

25.1.2 Project Metrics

Unlike software process metrics that are used for strategic purposes, software project measures are tactical. That is, project metrics and the indicators derived from them are used by a project manager and a software team to adapt project workflow and technical activities.

The first application of project metrics on most software projects occurs during estimation. Metrics collected from past projects are used as a basis from which effort and time estimates are made for current software work. As a project proceeds, measures of effort and calendar time expended are compared to original estimates (and the project schedule). The project manager uses these data to monitor and control progress.

As technical work commences, other project metrics begin to have significance. Production rates represented in terms of models created, review hours, function points, and delivered source lines are measured. In addition, errors uncovered during each software engineering task are tracked. As the software evolves from requirements into design, technical metrics (Chapter 23) are collected to assess design quality and to provide indicators that will influence the approach taken to code generation and testing.

The intent of project metrics is twofold. First, these metrics are used to minimize the development schedule by making the adjustments necessary to avoid delays and mitigate potential problems and risks. Second, project metrics are used to assess product quality on an ongoing basis and, when necessary, modify the technical approach to improve quality.

As quality improves, defects are minimized, and as the defect count goes down, the amount of rework required during the project is also reduced. This leads to a reduction in overall project cost.

? How should we use metrics during the project itself?

SafeHome



Establishing a Metrics Approach

The scene: Doug Miller's office as the SafeHome software project is about

to begin.

The players: Doug Miller (manager of the SafeHome software engineering team) and Vinod Raman and Jamie Lazar, members of the product software engineering team.

The conversation:

Doug: Before we start work on this project, I'd like you guys to define and collect a set of simple metrics. To start, you'll have to define your goals.

Vinod (frowning): We've never done that before, and . . .

Jamie (interrupting): And based on the time line management has been talking about, we'll never have the time. What good are metrics anyway?

Doug (raising his hand to stop the onslaught):

Slow down and take a breath, guys. The fact that we've never done it before is all the more reason to start now, and the metrics work I'm talking about shouldn't take much time at all . . . in fact, it just might save us time.

Vinod: How?

Doug: Look, we're going to be doing a lot more in-house software engineering as our products get more intelligent, become Web enabled, all that . . . and we need to understand the process we use to build software . . . and improve it so we can build software better. The only way to do that is to measure.

Jamie: But we're under time pressure, Doug. I'm not in favor of more paper pushing . . . we need the time to do our work, not collect data.

Doug (calmly): Jamie, an engineer's work involves collecting data, evaluating it, and using the results to improve the product and the process. Am I wrong?

Jamie: No, but . . .

Doug: What if we hold the number of measures we collect to no more than five or six and focus on quality?

Vinod: No one can argue against high quality . . .

Jamie: True . . . but, I don't know. I still think this isn't necessary.

Doug: I'm going to ask you to humor me on this one. How much do you guys know about software metrics?

Jamie (looking at Vinod): Not much.

Doug: Here are some Web refs . . . spend a few hours getting up to speed.

Jamie (smiling): I thought you said this wouldn't take any time.

Doug: Time you spend learning is never wasted . . . go do it and then we'll establish some goals, ask a few questions, and define the metrics we need to collect.

25.2 SOFTWARE MEASUREMENT

Note:

"Not everything that can be counted counts, and not everything that counts can be counted."

Albert Einstein

In Chapter 23, I noted that measurements in the physical world can be categorized in two ways: direct measures (e.g., the length of a bolt) and indirect measures (e.g., the "quality" of bolts produced, measured by counting rejects). Software metrics can be categorized similarly.

Direct measures of the software process include cost and effort applied. Direct measures of the product include lines of code (LOC) produced, execution speed, memory size, and defects reported over some set period of time. Indirect measures of the product include functionality, quality, complexity, efficiency, reliability, maintainability, and many other "–abilities" that are discussed in Chapter 14.

The cost and effort required to build software, the number of lines of code produced, and other direct measures are relatively easy to collect, as long as specific conventions for measurement are established in advance. However, the quality and functionality of software or its efficiency or maintainability are more difficult to assess and can be measured only indirectly.

I have partitioned the software metrics domain into process, project, and product metrics and noted that product metrics that are private to an individual are often combined to develop project metrics that are public to a software team. Project metrics are then consolidated to create process metrics that are public to the software organization as a whole. But how does an organization combine metrics that come from different individuals or projects?

To illustrate, consider a simple example. Individuals on two different project teams record and categorize all errors that they find during the software process. Individual measures are then combined to develop team measures. Team A found 342 errors during the software process prior to release. Team B found 184 errors. All other things being equal, which team is more effective in uncovering errors throughout the process? Because you do not know the size or complexity of the projects, you cannot answer this question. However, if the measures are normalized, it is



Because many factors affect software work, don't use metrics to compare individuals or teams.

FIGURE 25.2Size-oriented
metrics

Project	LOC	Effort	\$(000)	Pp. doc.	Errors	Defects	People
alpha	12,100	24	168	365	134	29	3
beta	27,200	62	440	1224	321	86	5
gamma	20,200	43	314	1050	256	64	6
•	•	•	•	•	•		
•	•	•	•	•	•		
•	•	•	•	•	•		

possible to create software metrics that enable comparison to broader organizational averages.

25.2.1 Size-Oriented Metrics

Size-oriented software metrics are derived by normalizing quality and/or productivity measures by considering the *size* of the software that has been produced. If a software organization maintains simple records, a table of size-oriented measures, such as the one shown in Figure 25.2, can be created. The table lists each software development project that has been completed over the past few years and corresponding measures for that project. Referring to the table entry (Figure 25.2) for project alpha: 12,100 lines of code were developed with 24 person-months of effort at a cost of \$168,000. It should be noted that the effort and cost recorded in the table represent all software engineering activities (analysis, design, code, and test), not just coding. Further information for project alpha indicates that 365 pages of documentation were developed, 134 errors were recorded before the software was released, and 29 defects were encountered after release to the customer within the first year of operation. Three people worked on the development of software for project alpha.

In order to develop metrics that can be assimilated with similar metrics from other projects, you can choose lines of code as a normalization value. From the rudimentary data contained in the table, a set of simple size-oriented metrics can be developed for each project:

- Errors per KLOC (thousand lines of code)
- Defects per KLOC
- \$ per KLOC
- Pages of documentation per KLOC

In addition, other interesting metrics can be computed:

- Errors per person-month
- KLOC per person-month
- \$ per page of documentation

KEY POINT

Size-oriented metrics are widely used, but debate about their validity and applicability continues.

Size-oriented metrics are not universally accepted as the best way to measure the software process. Most of the controversy swirls around the use of lines of code as a key measure. Proponents of the LOC measure claim that LOC is an “artifact” of all software development projects that can be easily counted, that many existing software estimation models use LOC or KLOC as a key input, and that a large body of literature and data predicated on LOC already exists. On the other hand, opponents argue that LOC measures are programming language dependent, that when productivity is considered, they penalize well-designed but shorter programs; that they cannot easily accommodate nonprocedural languages; and that their use in estimation requires a level of detail that may be difficult to achieve (i.e., the planner must estimate the LOC to be produced long before analysis and design have been completed).

25.2.2 Function-Oriented Metrics

Function-oriented software metrics use a measure of the functionality delivered by the application as a normalization value. The most widely used function-oriented metric is the function point (FP). Computation of the function point is based on characteristics of the software’s information domain and complexity. The mechanics of FP computation have been discussed in Chapter 23.³

The function point, like the LOC measure, is controversial. Proponents claim that FP is programming language independent, making it ideal for applications using conventional and nonprocedural languages, and that it is based on data that are more likely to be known early in the evolution of a project, making FP more attractive as an estimation approach. Opponents claim that the method requires some “sleight of hand” in that computation is based on subjective rather than objective data, that counts of the information domain (and other dimensions) can be difficult to collect after the fact, and that FP has no direct physical meaning—it’s just a number.

25.2.3 Reconciling LOC and FP Metrics

The relationship between lines of code and function points depends upon the programming language that is used to implement the software and the quality of the design. A number of studies have attempted to relate FP and LOC measures. The following table⁴ [QSM02] provides rough estimates of the average number of lines of code required to build one function point in various programming languages:

³ See Section 23.2.1 for a detailed discussion of FP computation.

⁴ Used with permission of Quantitative Software Management (www.qsm.com), copyright 2002.

LOC per Function Point

Programming Language	Average	Median	Low	High
Access	35	38	15	47
Ada	154	—	104	205
APS	86	83	20	184
ASP 69	62	—	32	127
Assembler	337	315	91	694
C	162	109	33	704
C++	66	53	29	178
Clipper	38	39	27	70
COBOL	77	77	14	400
Cool:Gen/IEF	38	31	10	180
Culprit	51	—	—	—
DBase IV	52	—	—	—
Easytrieve+	33	34	25	41
Excel47	46	—	31	63
Focus	43	42	32	56
FORTRAN	—	—	—	—
FoxPro	32	35	25	35
Ideal	66	52	34	203
IEF/Cool:Gen	38	31	10	180
Informix	42	31	24	57
Java	63	53	77	—
JavaScript	58	63	42	75
JCL	91	123	26	150
JSP	59	—	—	—
Lotus Notes	21	22	15	25
Mantis	71	27	22	250
Mapper	118	81	16	245
Natural	60	52	22	141
Oracle	30	35	4	217
PeopleSoft	33	32	30	40
Perl	60	—	—	—
PL/1	78	67	22	263
Powerbuilder	32	31	11	105
REXX	67	—	—	—
RPG II/III	61	49	24	155
SAS	40	41	33	49
Smalltalk	26	19	10	55
SQL	40	37	7	110
VBScript36	34	27	50	—
Visual Basic	47	42	16	158

A review of these data indicates that one LOC of C++ provides approximately 2.4 times the “functionality” (on average) as one LOC of C. Furthermore, one LOC of Smalltalk provides at least four times the functionality of an LOC for a conventional programming language such as Ada, COBOL, or C. Using the information contained in the table, it is possible to “backfire” [Jon98] existing software to estimate the number of function points, once the total number of programming language statements are known.

LOC and FP measures are often used to derive productivity metrics. This invariably leads to a debate about the use of such data. Should the LOC/person-month (or FP/person-month) of one group be compared to similar data from another? Should managers appraise the performance of individuals by using these metrics? The answer to these questions is an emphatic “No!” The reason for this response is that many factors influence productivity, making for “apples and oranges” comparisons that can be easily misinterpreted.

Function points and LOC-based metrics have been found to be relatively accurate predictors of software development effort and cost. However, in order to use LOC and FP for estimation (Chapter 26), an historical baseline of information must be established.

Within the context of process and project metrics, you should be concerned primarily with productivity and quality—measures of software development “output” as a function of effort and time applied and measures of the “fitness for use” of the work products that are produced. For process improvement and project planning purposes, your interest is historical. What was software development productivity on past projects? What was the quality of the software that was produced? How can past productivity and quality data be extrapolated to the present? How can it help us improve the process and plan new projects more accurately?

25.2.4 Object-Oriented Metrics

Conventional software project metrics (LOC or FP) can be used to estimate object-oriented software projects. However, these metrics do not provide enough granularity for the schedule and effort adjustments that are required as you iterate through an evolutionary or incremental process. Lorenz and Kidd [Lor94] suggest the following set of metrics for OO projects:

Number of scenario scripts. A scenario script (analogous to use cases discussed throughout Part 2 of this book) is a detailed sequence of steps that describe the interaction between the user and the application. Each script is organized into triplets of the form

{initiator, action, participant}

where **initiator** is the object that requests some service (that initiates a message), *action* is the result of the request, and **participant** is the server object that satisfies the request. The number of scenario scripts is directly correlated to the size of the



It is not uncommon for multiple-scenario scripts to mention the same functionality or data objects. Therefore, be careful when using script counts. Many scripts can sometimes be reduced to a single class or set of code.



Classes can vary in size and complexity. Therefore, it's worth considering classifying class counts by size and complexity.

application and to the number of test cases that must be developed to exercise the system once it is constructed.

Number of key classes. *Key classes* are the “highly independent components” [Lor94] that are defined early in object-oriented analysis (Chapter 6).⁵ Because key classes are central to the problem domain, the number of such classes is an indication of the amount of effort required to develop the software and also an indication of the potential amount of reuse to be applied during system development.

Number of support classes. *Support classes* are required to implement the system but are not immediately related to the problem domain. Examples might be user interface (GUI) classes, database access and manipulation classes, and computation classes. In addition, support classes can be developed for each of the key classes. Support classes are defined iteratively throughout an evolutionary process. The number of support classes is an indication of the amount of effort required to develop the software and also an indication of the potential amount of reuse to be applied during system development.

Average number of support classes per key class. In general, key classes are known early in the project. Support classes are defined throughout. If the average number of support classes per key class were known for a given problem domain, estimating (based on total number of classes) would be greatly simplified. Lorenz and Kidd suggest that applications with a GUI have between two and three times the number of support classes as key classes. Non-GUI applications have between one and two times the number of support classes as key classes.

Number of subsystems. A *subsystem* is an aggregation of classes that support a function that is visible to the end user of a system. Once subsystems are identified, it is easier to lay out a reasonable schedule in which work on subsystems is partitioned among project staff.

To be used effectively in an object-oriented software engineering environment, metrics similar to those noted above should be collected along with project measures such as effort expended, errors and defects uncovered, and models or documentation pages produced. As the database grows (after a number of projects have been completed), relationships between the object-oriented measures and project measures will provide metrics that can aid in project estimation.

25.2.5 Use-Case–Oriented Metrics

Use cases⁶ are used widely as a method for describing customer-level or business domain requirements that imply software features and functions. It would seem reasonable to use the use case as a normalization measure similar to LOC or FP.

⁵ We referred to key classes as *analysis classes* in Part 2 of this book.

⁶ Use cases are introduced in Chapters 5 and 6.

Like FP, the use case is defined early in the software process, allowing it to be used for estimation before significant modeling and construction activities are initiated. Use cases describe (indirectly, at least) user-visible functions and features that are basic requirements for a system. The use case is independent of programming language. In addition, the number of use cases is directly proportional to the size of the application in LOC and to the number of test cases that will have to be designed to fully exercise the application.

Because use cases can be created at vastly different levels of abstraction, there is no standard “size” for a use case. Without a standard measure of what a use case is, its application as a normalization measure (e.g., effort expended per use case) is suspect.

Researchers have suggested *use-case points* (UCPs) as a mechanism for estimating project effort and other characteristics. The UCP is a function of the number of actors and transactions implied by the use-case models and is analogous to the FP in some ways. If you have further interest, see [Cle06].

25.2.6 WebApp Project Metrics

The objective of all WebApp projects is to deliver a combination of content and functionality to the end user. Measures and metrics used for traditional software engineering projects are difficult to translate directly to WebApps. Yet, it is possible to develop a database that allows access to internal productivity and quality measures derived over a number of projects. Among the measures that can be collected are:

Number of static Web pages. Web pages with static content (i.e., the end user has no control over the content displayed on the page) are the most common of all WebApp features. These pages represent low relative complexity and generally require less effort to construct than dynamic pages. This measure provides an indication of the overall size of the application and the effort required to develop it.

Number of dynamic Web pages. Web pages with dynamic content (i.e., end-user actions or other external factors result in customized content displayed on the page) are essential in all e-commerce applications, search engines, financial applications, and many other WebApp categories. These pages represent higher relative complexity and require more effort to construct than static pages. This measure provides an indication of the overall size of the application and the effort required to develop it.

Number of internal page links. Internal page links are pointers that provide a hyperlink to some other Web page within the WebApp. This measure provides an indication of the degree of architectural coupling within the WebApp. As the number of page links increases, the effort expended on navigational design and construction also increases.

Number of persistent data objects. One or more persistent data objects (e.g., a database or data file) may be accessed by a WebApp. As the number of persistent data objects grows, the complexity of the WebApp also grows and the effort to implement it increases proportionally.

Number of external systems interfaced. WebApps must often interface with “backroom” business applications. As the requirement for interfacing grows, system complexity and development effort also increase.

Number of static content objects. Static content objects encompass static text-based, graphical, video, animation, and audio information that are incorporated within the WebApp. Multiple content objects may appear on a single Web page.

Number of dynamic content objects. Dynamic content objects are generated based on end-user actions and encompass internally generated text-based, graphical, video, animation, and audio information that are incorporated within the WebApp. Multiple content objects may appear on a single Web page.

Number of executable functions. An executable function (e.g., a script or applet) provides some computational service to the end user. As the number of executable functions increases, modeling and construction effort also increase.

Each of the preceding measures can be determined at a relatively early stage. For example, you can define a metric that reflects the degree of end-user customization that is required for the WebApp and correlate it to the effort expended on the project and/or the errors uncovered as reviews and testing are conducted. To accomplish this, you define

N_{sp} = number of Static Web pages

N_{dp} = number of Dynamic Web pages

Then,

$$\text{Customization index, } C = \frac{N_{dp}}{N_{dp} + N_{sp}}$$

The value of C ranges from 0 to 1. As C grows larger, the level of WebApp customization becomes a significant technical issue.

Similar WebApp metrics can be computed and correlated with project measures such as effort expended, errors and defects uncovered, and models or documentation pages produced. As the database grows (after a number of projects have been completed), relationships between the WebApp measures and project measures will provide indicators that can aid in project estimation.

SOFTWARE TOOLS

**Project and Process Metrics**

Objective: To assist in the definition, collection, evaluation, and reporting of software measures and metrics.

Mechanics: Each tool varies in its application, but all provide mechanisms for collecting and evaluating data that lead to the computation of software metrics.

Representative Tools:⁷

Function Point WORKBENCH, developed by Charismatek (www.charismatek.com.au), offers a wide array of FP-oriented metrics.

MetricCenter, developed by Distributive Software (www.distributive.com), supports automating data collection, analysis, chart formatting, report generation, and other measurement tasks.

PSM Insight, developed by Practical Software and Systems Measurement (www.psmisc.com), assists in the creation and subsequent analysis of a project measurement database.

SLIM tool set, developed by QSM (www.qsm.com), provides a comprehensive set of metrics and estimation tools.

SPR tool set, developed by Software Productivity Research (www.spr.com), offers a comprehensive collection of FP-oriented tools.

TychoMetrics, developed by Predicate Logic, Inc. (www.predicate.com), is a tool suite for management metrics collection and reporting.

25.3 METRICS FOR SOFTWARE QUALITY



Software is a complex entity. Therefore, errors are to be expected as work products are developed. Process metrics are intended to improve the software process so that errors are uncovered in the most effective manner.

The overriding goal of software engineering is to produce a high-quality system, application, or product within a time frame that satisfies a market need. To achieve this goal, you must apply effective methods coupled with modern tools within the context of a mature software process. In addition, a good software engineer (and good software engineering managers) must measure if high quality is to be realized.

The quality of a system, application, or product is only as good as the requirements that describe the problem, the design that models the solution, the code that leads to an executable program, and the tests that exercise the software to uncover errors. You can use measurement to assess the quality of the requirements and design models, the source code, and the test cases that have been created as the software is engineered. To accomplish this real-time assessment, you apply product metrics (Chapter 23) to evaluate the quality of software engineering work products in objective, rather than subjective ways.

A project manager must also evaluate quality as the project progresses. Private metrics collected by individual software engineers are combined to provide project-level results. Although many quality measures can be collected, the primary thrust at the project level is to measure errors and defects. Metrics derived from these measures provide an indication of the effectiveness of individual and group software quality assurance and control activities.

⁷ Tools noted here do not represent an endorsement, but rather a sampling of tools in this category. In most cases, tool names are trademarked by their respective developers.

Metrics such as work product errors per function point, errors uncovered per review hour, and errors uncovered per testing hour provide insight into the efficacy of each of the activities implied by the metric. Error data can also be used to compute the *defect removal efficiency* (DRE) for each process framework activity. DRE is discussed in Section 25.3.3.

25.3.1 Measuring Quality

Although there are many measures of software quality,⁸ correctness, maintainability, integrity, and usability provide useful indicators for the project team. Gilb [Gil88] suggests definitions and measures for each.

WebRef

An excellent source of information on software quality and related topics (including metrics) can be found at www.qualityworld.com.

Correctness. A program must operate correctly or it provides little value to its users. Correctness is the degree to which the software performs its required function. The most common measure for correctness is defects per KLOC, where a defect is defined as a verified lack of conformance to requirements. When considering the overall quality of a software product, defects are those problems reported by a user of the program after the program has been released for general use. For quality assessment purposes, defects are counted over a standard period of time, typically one year.

Maintainability. Software maintenance and support accounts for more effort than any other software engineering activity. Maintainability is the ease with which a program can be corrected if an error is encountered, adapted if its environment changes, or enhanced if the customer desires a change in requirements. There is no way to measure maintainability directly; therefore, you must use indirect measures. A simple time-oriented metric is *mean-time-to-change* (MTTC), the time it takes to analyze the change request, design an appropriate modification, implement the change, test it, and distribute the change to all users. On average, programs that are maintainable will have a lower MTTC (for equivalent types of changes) than programs that are not maintainable.

Integrity. Software integrity has become increasingly important in the age of cyber terrorists and hackers. This attribute measures a system's ability to withstand attacks (both accidental and intentional) to its security. Attacks can be made on all three components of software: programs, data, and documentation.

To measure integrity, two additional attributes must be defined: threat and security. *Threat* is the probability (which can be estimated or derived from empirical evidence) that an attack of a specific type will occur within a given time. *Security* is the probability (which can be estimated or derived from

⁸ A detailed discussion of the factors that influence software quality and the metrics that can be used to assess software quality has been presented in Chapter 23.

empirical evidence) that the attack of a specific type will be repelled. The integrity of a system can then be defined as:

$$\text{Integrity} = \Sigma [1 - (\text{threat} \times (1 - \text{security}))]$$

For example, if threat (the probability that an attack will occur) is 0.25 and security (the likelihood of repelling an attack) is 0.95, the integrity of the system is 0.99 (very high). If, on the other hand, the threat probability is 0.50 and the likelihood of repelling an attack is only 0.25, the integrity of the system is 0.63 (unacceptably low).

Usability. If a program is not easy to use, it is often doomed to failure, even if the functions that it performs are valuable. Usability is an attempt to quantify ease of use and can be measured in terms of the characteristics presented in Chapter 11.

The four factors just described are only a sampling of those that have been proposed as measures for software quality. Chapter 23 considers this topic in additional detail.

25.3.2 Defect Removal Efficiency

A quality metric that provides benefit at both the project and process level is *defect removal efficiency* (DRE). In essence, DRE is a measure of the filtering ability of quality assurance and control actions as they are applied throughout all process framework activities.

When considered for a project as a whole, DRE is defined in the following manner:

$$\text{DRE} = \frac{E}{E + D}$$



If DRE is low as you move through analysis and design, spend some time improving the way you conduct technical reviews.

where E is the number of errors found before delivery of the software to the end user and D is the number of defects found after delivery.

The ideal value for DRE is 1. That is, no defects are found in the software. Realistically, D will be greater than 0, but the value of DRE can still approach 1 as E increases for a given value of D . In fact, as E increases, it is likely that the final value of D will decrease (errors are filtered out before they become defects). If used as a metric that provides an indicator of the filtering ability of quality control and assurance activities, DRE encourages a software project team to institute techniques for finding as many errors as possible before delivery.

DRE can also be used within the project to assess a team's ability to find errors before they are passed to the next framework activity or software engineering action. For example, requirements analysis produces a requirements model that can be reviewed to find and correct errors. Those errors that are not found during the review of the requirements model are passed on to design (where they may or may not be found). When used in this context, we redefine DRE as

$$\text{DRE}_i = \frac{E_i}{E_i + E_{i+1}}$$

where E_i is the number of errors found during software engineering action i and E_{i+1} is the number of errors found during software engineering action $i + 1$ that are traceable to errors that were not discovered in software engineering action i .

A quality objective for a software team (or an individual software engineer) is to achieve DRE_i that approaches 1. That is, errors should be filtered out before they are passed on to the next activity or action.

SAFEHOME



Establishing a Metrics Approach

The scene: Doug Miller's office two days after initial meeting on software metrics.

The players: Doug Miller (manager of the *SafeHome* software engineering team) and Vinod Raman and Jamie Lazar, members of the product software engineering team.

The conversation:

Doug: You both had a chance to learn a little about process and project metrics?

Vinod and Jamie: [Both nod.]

Doug: It's always a good idea to establish goals when you adopt any metrics. What are yours?

Vinod: Our metrics should focus on quality. In fact, our overall goal is to keep the number of errors we pass on from one software engineering activity to the next to an absolute minimum.

Doug: And be very sure you keep the number of defects released with the product to as close to zero as possible.

Vinod (nodding): Of course.

Jamie: I like DRE as a metric, and I think we can use it for the entire project, but also as we move from one

framework activity to the next. It'll encourage us to find errors at each step.

Vinod: I'd also like to collect the number of hours we spend on reviews.

Jamie: And the overall effort we spend on each software engineering task.

Doug: You can compute a review-to-development ratio . . . might be interesting.

Jamie: I'd like to track some use-case data as well. Like the amount of effort required to develop a use case, the amount of effort required to build software to implement a use case, and . . .

Doug (smiling): I thought we were going to keep this simple.

Vinod: We should, but once you get into this metrics stuff, there's a lot of interesting things to look at.

Doug: I agree, but let's walk before we run and stick to our goal. Limit data to be collected to five or six items, and we're ready to go.

25.4 INTEGRATING METRICS WITHIN THE SOFTWARE PROCESS

The majority of software developers still do not measure, and sadly, most have little desire to begin. As I noted earlier in this chapter, the problem is cultural. Attempting to collect measures where none have been collected in the past often precipitates resistance. "Why do we need to do this?" asks a harried project manager. "I don't see the point," complains an overworked practitioner.

In this section, I consider some arguments for software metrics and present an approach for instituting a metrics collection program within a software engineering organization. But before I begin, some words of wisdom (now more than two decades old) are suggested by Grady and Caswell [Gra87]:

Some of the things we describe here will sound quite easy. Realistically, though, establishing a successful company-wide software metrics program is hard work. When we say that you must wait at least three years before broad organizational trends are available, you get some idea of the scope of such an effort.

The caveat suggested by the authors is well worth heeding, but the benefits of measurement are so compelling that the hard work is worth it.

25.4.1 Arguments for Software Metrics

Why is it so important to measure the process of software engineering and the product (software) that it produces? The answer is relatively obvious. If you do not measure, there is no real way of determining whether you are improving. And if you are not improving, you are lost.

By requesting and evaluating productivity and quality measures, a software team (and its management) can establish meaningful goals for improvement of the software process. Early in this book, I noted that software is a strategic business issue for many companies. If the process through which it is developed can be improved, a direct impact on the bottom line can result. But to establish goals for improvement, the current status of software development must be understood. Hence, measurement is used to establish a process baseline from which improvements can be assessed.

The day-to-day rigors of software project work leave little time for strategic thinking. Software project managers are concerned with more mundane (but equally important) issues: developing meaningful project estimates, producing higher-quality systems, getting product out the door on time. By using measurement to establish a project baseline, each of these issues becomes more manageable. I have already noted that the baseline serves as a basis for estimation. Additionally, the collection of quality metrics enables an organization to “tune” its software process to remove the “vital few” causes of defects that have the greatest impact on software development.⁹

25.4.2 Establishing a Baseline

By establishing a metrics baseline, benefits can be obtained at the process, project, and product (technical) levels. Yet the information that is collected need not be fundamentally different. The same metrics can serve many masters. The metrics baseline consists of data collected from past software development projects and can be as simple as the table presented in Figure 25.2 or as complex as a comprehensive database containing dozens of project measures and the metrics derived from them.

To be an effective aid in process improvement and/or cost and effort estimation, baseline data must have the following attributes: (1) data must be reasonably accurate—“guesstimates” about past projects are to be avoided, (2) data should be collected for as many projects as possible, (3) measures must be consistent (for example, a line of code must be interpreted consistently across all projects for which



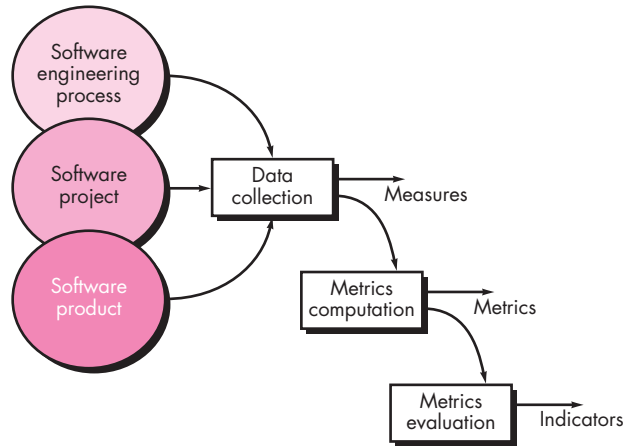
note:

“We manage things by the numbers in many aspects of our lives . . . These numbers give us insight and help steer our actions.”

**Michael Mah
and
Larry Putnam**

 **What is a metrics baseline and what benefit does it provide to a software engineer?**

⁹ These ideas have been formalized into an approach called *statistical software quality assurance*.

FIGURE 25.3**Software metrics collection process**

data are collected), (4) applications should be similar to work that is to be estimated—it makes little sense to use a baseline for batch information systems work to estimate a real-time, embedded application.

25.4.3 Metrics Collection, Computation, and Evaluation

The process for establishing a metrics baseline is illustrated in Figure 25.3. Ideally, data needed to establish a baseline have been collected in an ongoing manner. Sadly, this is rarely the case. Therefore, data collection requires an historical investigation of past projects to reconstruct required data. Once measures have been collected (unquestionably the most difficult step), metrics computation is possible. Depending on the breadth of measures collected, metrics can span a broad range of application-oriented metrics (e.g., LOC, FP, object-oriented, WebApp) as well as other quality- and project-oriented metrics. Finally, metrics must be evaluated and applied during estimation, technical work, project control, and process improvement. Metrics evaluation focuses on the underlying reasons for the results obtained and produces a set of indicators that guide the project or process.

25.5 METRICS FOR SMALL ORGANIZATIONS



If you're just starting to collect metrics data, remember to keep it simple. If you bury yourself with data, your metrics effort will fail.

The vast majority of software development organizations have fewer than 20 software people. It is unreasonable, and in most cases unrealistic, to expect that such organizations will develop comprehensive software metrics programs. However, it is reasonable to suggest that software organizations of all sizes measure and then use the resultant metrics to help improve their local software process and the quality and timeliness of the products they produce.

A commonsense approach to the implementation of any software process-related activity is: keep it simple, customize to meet local needs, and be sure it adds value.

? How should we derive a set of “simple” software metrics.

In the paragraphs that follow, I examine how these guidelines relate to metrics for small shops.¹⁰

“Keep it simple” is a guideline that works reasonably well in many activities. But how should you derive a “simple” set of software metrics that still provides value, and how can you be sure that these simple metrics will meet the specific needs of your software organization? You can begin by focusing not on measurement but rather on results. The software group is polled to define a single objective that requires improvement. For example, “reduce the time to evaluate and implement change requests.” A small organization might select the following set of easily collected measures:

- Time (hours or days) elapsed from the time a request is made until evaluation is complete, t_{queue} .
- Effort (person-hours) to perform the evaluation, W_{eval} .
- Time (hours or days) elapsed from completion of evaluation to assignment of change order to personnel, t_{eval} .
- Effort (person-hours) required to make the change, W_{change} .
- Time required (hours or days) to make the change, t_{change} .
- Errors uncovered during work to make change, E_{change} .
- Defects uncovered after change is released to the customer base, D_{change} .

Once these measures have been collected for a number of change requests, it is possible to compute the total elapsed time from change request to implementation of the change and the percentage of elapsed time absorbed by initial queuing, evaluation and change assignment, and change implementation. Similarly, the percentage of effort required for evaluation and implementation can be determined. These metrics can be assessed in the context of quality data, E_{change} and D_{change} . The percentages provide insight into where the change request process slows down and may lead to process improvement steps to reduce t_{queue} , W_{eval} , t_{eval} , W_{change} , and/or E_{change} . In addition, the defect removal efficiency can be computed as

$$\text{DRE} = \frac{E_{\text{change}}}{E_{\text{change}} + D_{\text{change}}}$$

DRE can be compared to elapsed time and total effort to determine the impact of quality assurance activities on the time and effort required to make a change.

For small groups, the cost of collecting measures and computing metrics ranges from 3 to 8 percent of project budget during the learning phase and then drops to less than 1 percent of project budget after software engineers and project managers have become familiar with the metrics program [Gra99]. These costs can show a

¹⁰ This discussion is equally relevant to software teams that have adopted an agile software development process (Chapter 3).

substantial return on investment if the insights derived from metrics data lead to meaningful process improvement for the software organization.

25.6 ESTABLISHING A SOFTWARE METRICS PROGRAM

The Software Engineering Institute has developed a comprehensive guidebook [Par96b] for establishing a “goal-driven” software metrics program. The guidebook suggests the following steps:

1. Identify your business goals.
2. Identify what you want to know or learn.
3. Identify your subgoals.
4. Identify the entities and attributes related to your subgoals.
5. Formalize your measurement goals.
6. Identify quantifiable questions and the related indicators that you will use to help you achieve your measurement goals.
7. Identify the data elements that you will collect to construct the indicators that help answer your questions.
8. Define the measures to be used, and make these definitions operational.
9. Identify the actions that you will take to implement the measures.
10. Prepare a plan for implementing the measures.

WebRef

A Guidebook for Goal-Driven Software Measurement can be downloaded from www.sei.cmu.edu.

A detailed discussion of these steps is best left to the SEI’s guidebook. However, a brief overview of key points is worthwhile.

Because software supports business functions, differentiates computer-based systems or products, or acts as a product in itself, goals defined for the business can almost always be traced downward to specific goals at the software engineering level. For example, consider the *SafeHome* product. Working as a team, software engineering and business managers develop a list of prioritized business goals:

1. Improve our customers’ satisfaction with our products.
2. Make our products easier to use.
3. Reduce the time it takes us to get a new product to market.
4. Make support for our products easier.
5. Improve our overall profitability.

The software organization examines each business goal and asks: “What activities do we manage, execute, or support and what do we want to improve within these activities?” To answer these questions the SEI recommends the creation of an “entity-question list” in which all things (entities) within the software process that are managed or influenced by the software organization are noted. Examples of

KEY POINT

The software metrics you choose should be driven by the business and technical goals you wish to accomplish.

entities include development resources, work products, source code, test cases, change requests, software engineering tasks, and schedules. For each entity listed, software people develop a set of questions that assess quantitative characteristics of the entity (e.g., size, cost, time to develop). The questions derived as a consequence of the creation of an entity-question list lead to the derivation of a set of sub-goals that relate directly to the entities created and the activities performed as part of the software process.

Consider the fourth goal: “Make support for our products easier.” The following list of questions might be derived for this goal [Par96b]:

- Do customer change requests contain the information we require to adequately evaluate the change and then implement it in a timely manner?
- How large is the change request backlog?
- Is our response time for fixing bugs acceptable based on customer need?
- Is our change control process (Chapter 22) followed?
- Are high-priority changes implemented in a timely manner?

Based on these questions, the software organization can derive the following sub-goal: *Improve the performance of the change management process*. The software process entities and attributes that are relevant to the subgoal are identified, and the measurement goals associated with them are delineated.

The SEI [Par96b] provides detailed guidance for steps 6 through 10 of its goal-driven measurement approach. In essence, you refine measurement goals into questions that are further refined into entities and attributes that are then refined into metrics.



Establishing a Metrics Program

The Software Productivity Center (www.spc.ca) suggests an eight-step approach for establishing a metrics program within a software organization that can be used as an alternative to the SEI approach described in Section 25.6. Their approach is summarized in this sidebar.

1. Understand the existing software process.
Framework activities (Chapter 2) are identified.
Input information for each activity is described.
Tasks associated with each activity are defined.
Quality assurance functions are noted.
Work products that are produced are listed.
2. Define the goals to be achieved by establishing a metrics program
Examples: improve accuracy of estimation,
improve product quality.
3. Identify metrics required to achieve goals.
Questions to be answered are defined; for example, *How many errors found in one framework activity can be traced to the preceding framework activity?*
Create measures and metrics that will help answer these questions.
4. Identify the measures and metrics to be collected and computed.
5. Establish a measurement collection process by answering these questions:
What is the source of the measurements?
Can tools be used to collect the data?
Who is responsible for collecting the data?
When are data collected and recorded?
How are data stored?

INFO

What validation mechanisms are used to ensure that the data are correct?

6. Acquire appropriate tools to assist in collection and assessment.
7. Establish a metrics database.

The relative sophistication of the database is established.

Use of related tools (e.g., an SCM repository, Chapter 26) is explored.

Existing database products are evaluated.

8. Define appropriate feedback mechanisms.
Who requires ongoing metrics information?
How is the information to be delivered?
What is the format of the information?

A considerably more detailed description of these eight steps can be downloaded from www.spc.ca/resources/metrics/.

25.7 SUMMARY

Measurement enables managers and practitioners to improve the software process; assist in the planning, tracking, and control of software projects; and assess the quality of the product (software) that is produced. Measures of specific attributes of the process, project, and product are used to compute software metrics. These metrics can be analyzed to provide indicators that guide management and technical actions.

Process metrics enable an organization to take a strategic view by providing insight into the effectiveness of a software process. Project metrics are tactical. They enable a project manager to adapt project workflow and technical approach in a real-time manner.

Both size- and function-oriented metrics are used throughout the industry. Size-oriented metrics use the line of code as a normalizing factor for other measures such as person-months or defects. The function point is derived from measures of the information domain and a subjective assessment of problem complexity. In addition, object-oriented metrics and Web application metrics can be used.

Software quality metrics, like productivity metrics, focus on the process, the project, and the product. By developing and analyzing a metrics baseline for quality, an organization can correct those areas of the software process that are the cause of software defects.

Measurement results in cultural change. Data collection, metrics computation, and metrics analysis are the three steps that must be implemented to begin a metrics program. In general, a goal-driven approach helps an organization focus on the right metrics for its business. By creating a metrics baseline—a database containing process and product measurements—software engineers and their managers can gain better insight into the work that they do and the product that they produce.

PROBLEMS AND POINTS TO PONDER

25.1. Describe the difference between process and project metrics in your own words.

25.2. Why should some software metrics be kept “private”? Provide examples of three metrics that should be private. Provide examples of three metrics that should be public.

25.3. What is an indirect measure, and why are such measures common in software metrics work?

25.4. Grady suggests an etiquette for software metrics. Can you add three more rules to those noted in Section 25.1.1?

25.5. Team A found 342 errors during the software engineering process prior to release. Team B found 184 errors. What additional measures would have to be made for projects A and B to determine which of the teams eliminated errors more efficiently? What metrics would you propose to help in making the determination? What historical data might be useful?

25.6. Present an argument against lines of code as a measure for software productivity. Will your case hold up when dozens or hundreds of projects are considered?

25.7. Compute the function point value for a project with the following information domain characteristics:

- Number of user inputs: 32
- Number of user outputs: 60
- Number of user inquiries: 24
- Number of files: 8
- Number of external interfaces: 2

Assume that all complexity adjustment values are average. Use the algorithm noted in Chapter 23.

25.8. Using the table presented in Section 25.2.3, make an argument against the use of assembler language based on the functionality delivered per statement of code. Again referring to the table, discuss why C++ would present a better alternative than C.

25.9. The software used to control a photocopier requires 32,000 lines of C and 4,200 lines of Smalltalk. Estimate the number of function points for the software inside the copier.

25.10. A Web engineering team has built an e-commerce WebApp that contains 145 individual pages. Of these pages, 65 are dynamic; that is, they are internally generated based on end-user input. What is the customization index for this application?

25.11. A WebApp and its support environment have not been fully fortified against attack. Web engineers estimate that the likelihood of repelling an attack is only 30 percent. The system does not contain sensitive or controversial information, so the threat probability is 25 percent. What is the integrity of the WebApp?

25.12. At the conclusion of a project, it has been determined that 30 errors were found during the modeling activity and 12 errors were found during the construction activity that were traceable to errors that were not discovered in the modeling activity. What is the DRE for the modeling activity?

25.13. A software team delivers a software increment to end users. The users uncover eight defects during the first month of use. Prior to delivery, the software team found 242 errors during formal technical reviews and all testing tasks. What is the overall DRE for the project after one month's usage?

FURTHER READINGS AND INFORMATION SOURCES

Software process improvement (SPI) has received a significant amount of attention over the past two decades. Since measurement and software metrics are key to successfully improving the software process, many books on SPI also discuss metrics. Rico (*ROI of Software Process Improvement*, J. Ross Publishing, 2004) provides an in-depth discussion of SPI and the metrics that can help an organization achieve it. Ebert and his colleagues (*Best Practices in Software Measurement*, Springer, 2004) address the use of measurement within the context of ISO and CMMI standards. Kan (*Metrics and Models in Software Quality Engineering*, 2d ed., Addison-Wesley, 2002) presents a collection of relevant metrics.

Ebert and Dumke (*Software Measurement*, Springer, 2007) provide a useful treatment of measurement and metrics as they should be applied for IT projects. McGarry and his colleagues (*Practical Software Measurement*, Addison-Wesley, 2001) present in-depth advice for assessing the software process. A worthwhile collection of papers has been edited by Haug and his colleagues (*Software Process Improvement: Metrics, Measurement, and Process Modeling*, Springer-Verlag, 2001). Florac and Carlton (*Measuring the Software Process*, Addison-Wesley, 1999) and Fenton and Pfleeger (*Software Metrics: A Rigorous and Practical Approach*, Revised, Brooks/Cole Publishers, 1998) discuss how software metrics can be used to provide the indicators necessary to improve the software process.

Laird and Brennan (*Software Measurement and Estimation*, Wiley-IEEE Computer Society Press, 2006) and Goodman (*Software Metrics: Best Practices for Successful IT Management*, Rothstein Associates, Inc., 2004) discuss the use of software metrics for project management and estimation. Putnam and Myers (*Five Core Metrics*, Dorset House, 2003) draw on a database of more than 6000 software projects to demonstrate how five core metrics—time, effort, size, reliability, and process productivity—can be used to control software projects. Maxwell (*Applied Statistics for Software Managers*, Prentice-Hall, 2003) presents techniques for analyzing software project data. Munson (*Software Engineering Measurement*, Auerbach, 2003) discusses a broad array of software engineering measurement issues. Jones (*Software Assessments, Benchmarks and Best Practices*, Addison-Wesley, 2000) describes both quantitative measurement and qualitative factors that help an organization assess its software process and practices.

Function point measurement has become a widely used technique in many areas of software engineering work. Parthasarathy (*Practical Software Estimation: Function Point Methods for Insourced and Outsourced Projects*, Addison-Wesley, 2007) provide a comprehensive guide. Garmus and Herron (*Function Point Analysis: Measurement Practices for Successful Software Projects*, Addison-Wesley, 2000) discuss process metrics with an emphasis on function point analysis.

Relatively little has been published on metrics for Web engineering work. However, Kaushik (*Web Analytics: An Hour a Day*, Sybex, 2007), Stern (*Web Metrics: Proven Methods for Measuring Web Site Success*, Wiley, 2002), Inan and Kean (*Measuring the Success of Your Website*, Longman, 2002), and Nobles and Grady (*Web Site Analysis and Reporting*, Premier Press, 2001) address Web metrics from a business and marketing perspective.

The latest research in the metrics area is summarized by the IEEE (*Symposium on Software Metrics*, published yearly). A wide variety of information sources on the process and project metrics is available on the Internet. An up-to-date list of World Wide Web references relevant to process and project metrics can be found at the SEPA website: **www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm**.